

Scenario 01 — PortPilot Maritime

A deliberately vulnerable web application for use in the CyberFort cyber range. Trainees use the CyberFort platform to scan, assess, and produce a CRA readiness report for the target.

What's in the box

```
01-portpilot-maritime/
├─ README.md           ← this file (deployment)
├─ docker-compose.yml
├─ .env.example
├─ app/
│   └─ Dockerfile
│   └─ requirements.txt ← pinned to known-vulnerable versions for OSV
│   └─ run.py
│   └─ portpilot/       ← Flask source (intentionally vulnerable)
├─ db/
│   └─ init.sql         ← seed users, vessels, cargo manifests
├─ docs/
│   └─ TRAINEE_HANDBOOK.md ← step-by-step walkthrough for the learner
│   └─ INSTRUCTOR_GUIDE.md ← answer key, scoring, patches
│   └─ CRA_CONTROL_MAPPING.md ← CRA Annex I evidence per finding
```

Topology

- **VM1** runs CyberFort (frontend :5173, backend :8000, scanners on :8010-:8013).
- **VM2** runs this scenario stack. The cyber range networks VM1 and VM2 on the same subnet.

Deploy on VM2

```
# Requires: docker engine + compose plugin
git clone <repo-url> /srv/portpilot-scenario
cd /srv/portpilot-scenario/01-portpilot-maritime
docker compose up -d --build
```

Wait ~15 s for Postgres to become healthy, then verify:

```
docker compose ps
curl -s http://localhost:8080/healthz # {"status":"ok"}
```

The Flask app listens on **8080/tcp** and (deliberately) PostgreSQL is exposed on **5432/tcp**.

Reset between trainee sessions

```
cd /srv/portpilot-scenario/01-portpilot-maritime
docker compose down -v
docker compose up -d --build
```

The `-v` removes the Postgres volume so the seed re-runs.

What the trainee should see

- CyberFort's **Nmap** scan of `<VM2-IP>` reports `8080/http` and `5432/postgresql` open.
- CyberFort's **ZAP** scan of `http://<VM2-IP>:8080` reports SQL injection on `/login`, reflected XSS on `/vessels`, and missing-auth on `/admin/manifests`.
- CyberFort's **Semgrep** scan of the tarred source reports hardcoded credentials in `config.py` and tainted SQL in `app.py`.
- CyberFort's **OSV** scan of `requirements.txt` reports advisories on Flask, Werkzeug, Jinja2, and requests.

Full walk-through and remediation are in [docs/TRAINEE_HANDBOOK.md](#) and [docs/INSTRUCTOR_GUIDE.md](#).

Safety

This image is intentionally vulnerable. **Do not deploy on a network that is reachable from the public internet or from a corporate LAN.** The cyber range network must be isolated.